VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

Projektová dokumentace

K projektu do předmětu IFJ a IAL – ifj2023

1. prosince 2023

Řešitelský tým:	*xseibe00* – varianta vv-BVS (bez rozšíření)	
Složení týmu:	**Seibert Jan (250788)**	(25 %)
	Honza Dominik (250585)	(25 %)
	Kotek Jiří (247561)	(25 %)
	Nepraš David (250720)	(25 %)

Obsah

Úvod 2
Komunikace a organizace 2
Návrh a implementace jednotlivých částí 2
Lexikální analýza 2
Syntaktická analýza 3
Prediktivní syntaktická analýza 3
Sémantická analýza 4
Tabulka symbolů 4
Generátor 4
Obecné informace 4
Začátek generování 4
Konec generování 4
Funkce 5
Deklarace funkce 5
Vestavěné funkce 5
Generování if a while 5
Deklarace proměnných 6
Generování výrazů 6
Zhodnocení a závěr 6
Přílohy 7
Obrázek 1,- Stavy konečného automatu lexikální analýzy 7
Obrázek 1 Výčet jednotlivých stavů konečného automatu 7
Tabulka 1, LL gramatika 8
Tabulka 3, Precedenční tabulka 9
Tabulka 2, LL tabulka 9

Úvod

Zadáním byla implementace překladače jazyka IFJ23, který bude obsahovat syntaktickou a sémantickou analýzu a vstupní řídící program převede do mezikódu IFJCode23.

Komunikace a organizace

Pro řízení vývoje jsme používali nástroj JIRA a pro verzování Bitbucket. Práci na hlavních částí překladače jsme si přerozdělili hned po zadání projektu následovně:

- Jiří Kotek – Lexikální analýza
- Jan Seibert – Syntaktická analýza
- David Nepraš – Sémantická analýza
- Dominik Honza – Generování mezikódu

Návrh a implementace jednotlivých částí

Lexikální analýza

Lexikální analyzátor je implementován pomocí konečného automatu[obr. 1]. Hlavním souborem je `scanner.c`, ve kterém se nachází klíčová funkce `GetToken()`, která nejprve zinicializuje celou strukturu tokenu, provede analýzu aktuálně zpracovávaného výrazu, a v případě konce řádku nastaví příznak `FollowingEOL` na hodnotu `true`. Struktura tokenu je tvořena:

- `unsigned int ID` – přiřazuje každému tokenu číslo pod kterým lze vyhledat
- `fluids* Fluidpayload` – struktura, slouží k uložení znakových a číselných řetězců
- `long long int value` – slouží k uložení celočíselné hodnoty
- `double floatingValue` – složí k uložení desetinného čísla nebo čísla s exponentem
- `tokenType type` – Přiřazení stavu koncového stavu konečného automatu
- `bool FollowingEOL` – Nastaví hodnotu true v případě, že za tokenem následuje konec řádku

Pomocná struktura `fluids* Fluidpayload` se nachází ve vlastním souboru `fluid_string.c`. Jedná se o dynamické pole, které se využívá při ukládání znakových řetězců a číselných hodnot. K poli přistupujeme pomocí funkce `AddCharToToken()`, která zajišťuje korektní přidání symbolu do řetězce. V souboru se nachází i další pomocné funkce pro práci s řetězci uvnitř tokenu.

Výchozím stavem automatu je stav `START`. Před vstupem do smyčky `while`, uvnitř které se nachází hlavní `switch` (pomocí kterého je implementován konečný automat), kontrolujeme přítomnost konce souboru (`EOF`) a komentářů. V případě odhalení lexikální chyby scanner vrací hodnotu parseru `LEX_ERR`. Pro korektní uvolnění všech alokovaných zdrojů tokenu se volá funkce `FlushToken()`.

Syntaktická analýza

Analyzování začíná zavoláním funkce `parse()` v souboru `parser.c`. V tomhle souboru je implementovaná celá syntaktická analýza shoru dolů a zároveň i část sémantické analýzy.

Jednotlivé stavy LL gramatiky [tab. 1, 2] jsou reprezentovaný stejnojmennou funkcí.

Se scannerem se komunikuje prostřednictvím funkce `GetToken()` v souboru `scanner.c`.

Jelikož je běžné, že je potřeba již načtený token vrátit zpět ke zpracování jiným stavem, je v globálních datech parseru `ParserDataT` definovaný atribut `bool pushBack`, který zajišťuje tohle správné chování.

Prediktivní syntaktická analýza

Z určitých stavů je potřeba přejít do vyhodnocování výrazů. Rozhodnutí o tom, jestli má dojít k přechodu do vyhodnocovací funkce se provádí načtením potřebného počtu tokenů předem. Aby byla zachována konzistence, tyto předem načtené tokeny jsou po rozhodnutí případně předány parametrem v poli do funkce `parseExpression()`, která se stará o analýzu výrazů.

Zpracování výrazů tedy probíhá v souboru `expr.c` a vstupní funkce je `parseExpression()`. Syntaktická analýza výrazů je řízena pomocí precedenční tabulky [tabulka 3]. Pro zjednodušení jsou některé operátory sjednoceny, například všechny relační operátory se chovají syntakticky stejně a není potřeba je v tabulce nějak rozlišovat.

V jazyce IFJ23 je možné zapisovat výraz přes více řádků. Proto analýza žádá neustále o nový token než nenarazí na nevalidní token pro výraz. Po téhle situaci je potřeba řádně rozlišit, jestli tento nevalidní token je už na jiném řádku než zpracovávaný výraz, nebo ne. Pokud je odřádkovaný, nejedná se o syntaktickou chybu a řízení je prováděno dále pomocí LL gramatiky a pokud není odřádkovaný jedná se o syntaktickou chybu. Kvůli této problematice je při vyhodnocování uchovávaná informace o následování `EOL` u předchozího tokenu. Jestli po posledním tokenu výrazu následuje `EOL` je předáváno zpět proměnnou `bool followingMlExprEOL` definovanou v globálních datech `ParserDataT`.

Problematické je u zpracovávání i uzavírací závorka. Jelikož za posledním argumentem funkce může následovat uzavírací závorka, je nutné tuhle situaci také rozlišit.

Proto se do funkce `parseExpression()` parametrem mimo vstupní pole tokenů předává i hodnota, zdali se jedná o argument funkce nebo ne – `bool isArg`. Pokud se narazí na vstupní token uzavírací závorky a `isArg` je `true`, vstupní token je přeměněn na uzavírací vstupní token `$`.

Zde je ještě potřeba zmínit, že otevírací závorku argumentu funkce není považována za součást výrazu.

Pokud bychom naopak narazili na uzavírací závorku a `isArg` by bylo rovno `false`, jedná se o neplatnou sekvenci symbolů a analýza je vyhodnocena jako syntaktická chyba.

Sémantická analýza

Sémantická analýza, úzce spojená se syntaktickou analýzou, je implementována v souboru `parser.c`. Celkový průběh procesu lze rozdělit do dvou hlavních průchodů kódu. Jako první se naplní globální tabulka symbolů funkcemi a následně se spustí syntaktická analýza. Současně s touto analýzou se vytvářejí lokální tabulky symbolů pro každý rozsah platnosti, kam jsou ukládány definice jednotlivých proměnných. Během tohoto průchodu probíhají některé důležité kontroly, jako ověření typů ve výrazech, kontroly návratových typů funkcí a detekce případných redefinicí proměnných.

Ve druhém průchodu, iniciovaném funkcí `sem_anal()`, probíhá kontrola definice a inicializace proměnných. Zároveň se provádí důkladné ověření správnosti volání funkcí, včetně kontroly počtu a typů parametrů. Tento proces zajistí konzistentní a bezchybnou sémantiku programu, přičemž se věnuje specifickým aspektům, jako jsou definice, inicializace a použití proměnných a funkcí v rámci kódu.

Největší problém při implementaci představoval neobvyklý způsob deklarace a volání funkcí. Avšak po důkladném pročtení zadání a pochopení problematiky byl tento problém zdárně vyřešen.

Tabulka symbolů

V našem řešení je tabulka symbolů implementována pomocí výškově vyváženého binárního vyhledávacího stromu v souboru `symtable.c`. Pro práci s tabulkami se používají funkce `avl_insert()` a `avl_search()`, které umožňují snadné vkládání a vyhledávání symbolů. Struktura automaticky udržuje výškovou vyváženost pomocí levých a pravých rotací.

Globální tabulka a všechny lokální tabulky jsou během průběhu sémantické analýzy uloženy v poli, přičemž globální tabulka je na indexu 0.

Generátor

Obecné informace

Generátor vytváří tříadresný mezikód jazyka IFJCode23. Ten je postupně generování za běhu syntaktické a sémantické analýzy. Generovaný kód je ukládán do vnitřní sdílené paměti programu dále `gData`. Po ukončení úspěšné syntaktické a sémantické analýzy, tj. nenastanou žádné kompilační chyby, dochází k zápisu generovaného kódu na standartní výstup.

Generátor je naprogramován jako samostatný výčet funkcí. Je ovšem závislí na souboru `fluid_string.h`, který zajištuje korektní manipulaci s sídelnou pamětí a zápisem do ní. Rozhraní generátoru je tvořena několika skupinami funkcí blízkých si předpokládaným využitím např. if, deklarace, funkce.

Generátor sám o sobě neobsahuje logiku, kdy se má daná funkce provolat. Včasné provolání vhodné funkce generátoru je zajištěno v syntaktické a sémantické analýze. Volání funkcí se provádí výhradně z tří míst kompilátoru: `main.c, parser.c, expr.c.`

Začátek generování

Generování začíná již před začátkem analýz předgenerováním hlavičky souboru, pomocných globálních proměnných, vestavěných funkcí, pomocných vestavěných funkcí a následně záhlavím funkce main.

Konec generování

Generování končí vygenerováním konce funkce main.

Funkce

Generované funkce se dělí do tří typů vestavěné funkce, vestavěné pomocné funkce a uživatelské funkce.

Deklarace funkce

- Vestavěná funkce začíná návěštím `$id_funkce`.
- Uživatelská funkce začíná návěštím `$_skip_id_funkce`.

Ve funkci je vytvořen nový rámec. Následuje definice parametrů funkce a přiřazení hodnoty ze zásobníku. Tělo je zpracováváno jako výraz a je zakončeno přiřazením do návratové proměnné `GF@_return`. Funkce končí zrušením rámce a návratem za své volání v kódu.

Pozn. Každá uživatelská funkce je obohacena o předsunutou instrukci `JUMP $_skip_id_funkce` a návěští za funkcí `LABEL $_skip_id_funkce`, za účelem možnosti deklarace kdekoliv v programu.

Volání funkce

Při volání funkce se parametry předávají skrze zásobník a vyvolá se daná funkce. Po dokončení se uloží její návratová hodnota stejně jako hodnota výrazu, aby ji bylo možné uložit.

Vestavěné funkce

Generátor podporuje tyto vestavěné funkce:

- `func length(_` s `: String) -> Int`
- `func substring(of` s `: String, startingAt` i `: Int, endingBefore` j `: Int)-> String?`
- `func readInt() -> Int?`
- `func readString() -> String?`
- `func readDouble() -> Double?`
- `func Double2Int(_ term : Double) -> Int`
- `func Int2Double(_ term : Int) -> Double`
- `func ord(_` c `: String) -> Int`
- `func chr(_` i `: Int) -> String`
- `func write ( term1 , term2 , …, term`n `)`

Tyto funkce jsou převedeny do tříadresného kodu a předgenerovány do `gData` za hlavičku generovaného kódu. Podrobnější definice funkcí jsou uvedeny v zadání.

Generování if a while

Generování if a while je rozděleno na skokovou logiku a těla.

- `If` je prvně vygenerován výraz podmínky následně skok podle výsledku výrazu na true tělo a false tělo.
- `While` je prvně vygenerováno návěští začátku while poté výraz podmínky, následně skok podle výsledku výrazu. Skok se provede při splnění podmínky za true tělo while. Při nesplnění se porvede tělo a skok na počáteční návěští while.

Návěští se označují `$_(skip,while,else,start..)_id`

`ID` se používá pro rozlišení návěští napříč generátorem, každý konstrukt má jedno `ID`, které je unikátní.

Deklarace proměnných

Deklarace probíhá vygenerováním přiřazovaného výrazu, vytvořením proměnné a přiřazením výrazu.

Proměnné jsou pojmenovány `LF@_id` kde id je identifikátor proměnné.

Generování výrazů

Generování výrazů probíhá voláním ze souboru `expr.c`. Každý výraz je vyhodnocován na zásobníku a po dokončení se jeho hodnota přiřadí do proměnné `GF@_exp`.

Při vyhodnocován se využívá pomocná vestavěná funkce `comparetypes()`. Ta zajistí korektní datové typy operandů. Při nutnosti zkonvertuje na požadovaný typ.

Zhodnocení a závěr

Závěr týmového projektu nám poskytuje příležitost reflektovat náročnost a výzvy, kterým jsme během úkolu čelili. Realizace kompilátoru patřila k těm složitějším projektům tohoto semestru, který vyžadoval nejen hlubší porozumění principům jazyka IFJ23, ale i schopnost efektivní spolupráce v týmu.

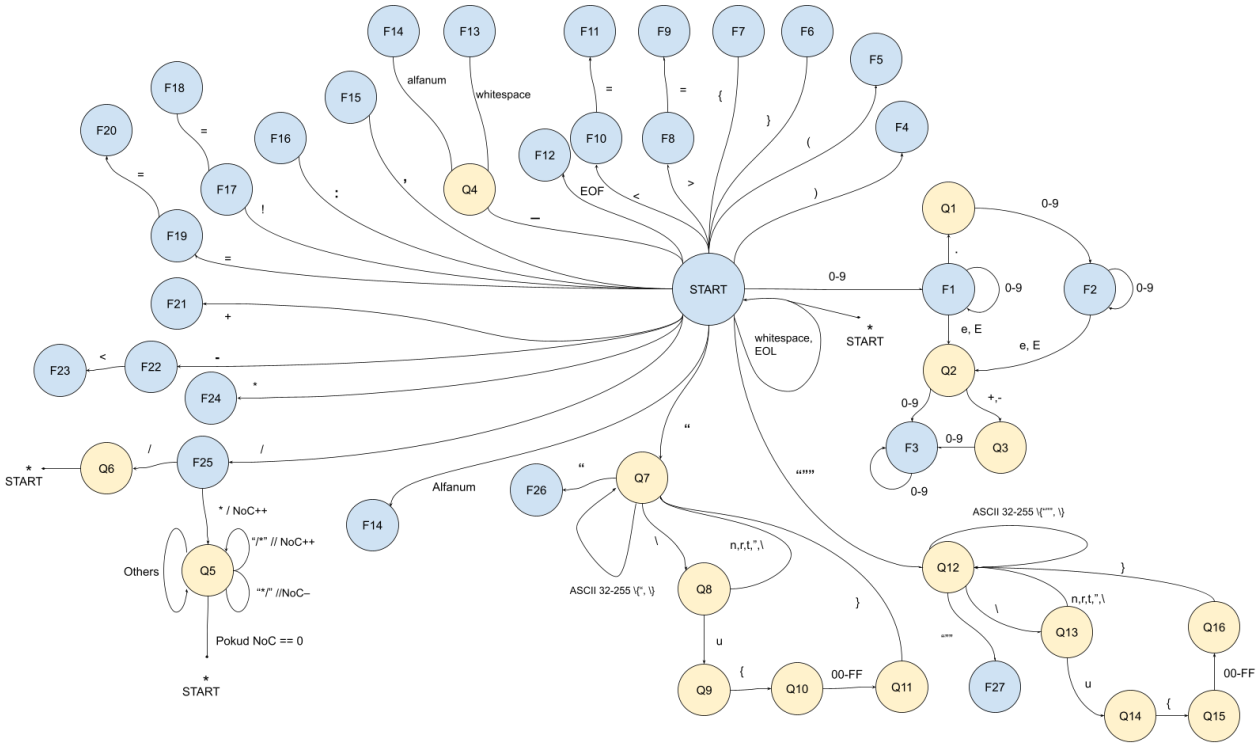
Během projektu jsme čelili řadě technických výzev spojených s analýzou, syntézou a optimalizací kódu. Komplexnost syntaxe a široké spektrum jazykových konstrukcí představovaly výzvu pro tvorbu robustního a efektivního kompilátoru.

Rozsahová náročnost projektu se projevila také v potřebě neustálé komunikace a koordinace v rámci týmu. Efektivní rozdělení úkolů, pravidelné kontroly a společné řešení problémů byly klíčové pro dosažení kvalitního výsledku. Zároveň jsme se naučili adaptovat na často se měnící požadavky projektu, vznikajících neúplným porozuměním zadání.

Celkově lze konstatovat, že náš týmový projekt byl pro každého člena osobní výzvou, ale i významným krokem směrem k hlubšímu porozumění fungování programovacích jazyků a kompilátorů.

Přílohy

Obrázek 1,- *Stavy konečného automatu lexikální analýzy*



Obrázek 1 Výčet jednotlivých stavů konečného automatu

F1	WHOLENUMBER		F17	EXCLAMATION		Q1	READING_DECIMAL_VAL
F2	DECIMALNUMBER_DEC		F18	NEQ		Q2	READING_EXPONENT_VAL
F3	DECIMALNUMBER_EXP		F19	EQ		Q3	READING_EXPONENT_SIGN_VAL
F4	CLBR		F20	EQEQ		Q4	UNDERSCORE_ID
F5	OPBR		F21	PLUS		Q5	MULTICOMMENT_COMMENT
F6	CLCB		F22	MINUS		Q6	SINGLE_LINE_COMMENT
F7	OPCB		F23	LAMBDA		Q7	READING_STRING
F8	GREATER		F24	MUL		Q8	START_ESCAPE
F9	GREATEREQ		F25	DIV		Q9	END_ESCAPE
F10	LESSER		F26	STRINGVALUE		Q10	HEX_ESCAPE
F11	LESSEREQ		F27	MULTILINE_STRINGVALUE		Q11	END_HEX_ESCAPE
F12	EOFILE					Q12	READING_MULTISTRING
F13	UNDERSCORE					Q13	START_ESCAPE_MULTI
F14	IDENTIFICATOR					Q14	END_ESCAPE_MULTI
F15	COMA					Q15	HEX_ESCAPE_MULTI
F16	DOUBLEDOT					Q16	END_HEX_ESCAPE_MULTI

Tabulka 1, LL gramatika

#	CUR. STATE		NEXT STATE	FIRST	FOLLOW
1.	<START>	->	<STAT> <START>	{ **if**, **while**, **let**, **var**, **id**, **func**, **EOL**, **EOF**}	{ **$** }
2.	<START>	->	**func** id (<PL>) <FUN_TYPE> { <FUN_BODY> } EOL <START>		
3.	<START>	->	**EOL** <START>		
4.	<START>	->	**EOF**		
5.	<STAT>	->	**if** <IF_H> { <STAT_LIST> } **else** { <STAT_LIST> } **EOL**	{ **if**, **while**, **let**, **var**, **id** }	{ **if**, **while**, **let**, **var**, **id**, **func**, **EOL**, **EOF**, **return** }
6.	<STAT>	->	**while (**<EXPR>) { <STAT_LIST> } **EOL**		
7.	<STAT>	->	<VAR_DEC> id <VAR_DEF> **EOL**		
8.	<STAT>	->	id <FUN_VAR> **EOL**		
9.	<FUN_VAR>	->	= <VALUE>	{ **=**, **(** }	{ **EOL** }
10.	<FUN_VAR>	->	(<P_VALUE> <ARGS2>)		
11.	<ARGS>	->	ε	{ **(**, **ε** }	{ **EOL**, **,** , **)** }
12.	<ARGS>	->	(<P_VALUE> <ARGS2>)		
13.	<ARGS2>	->	ε	{ **,** , **ε** }	{ **)** }
14.	<ARGS2>	->	, <P_VALUE> <ARGS2>		
15.	<STAT_LIST>	->	ε	{ **if**, **while**, **let**, **var**, **id**, **return**, **ε** }	{ **}** }
16.	<STAT_LIST>	->	**return** <VALUE>		
17.	<STAT_LIST>	->	<STAT> <STAT_LIST>		
18.	<PL>	->	ε	{ **_**, **id**, **ε** }	{ **)** }
19.	<PL>	->	<TERM> : <DATA_TYPE> <PL2>		
20.	<PL2>	->	ε	{ **,** , **ε** }	{ **)** }
21.	<PL2>	->	, <TERM> : <DATA_TYPE> <PL2>		
22.	<TERM>	->	_ id	{ **_**, **id** }	{ **:** }
23.	<TERM>	->	id id		
24.	<FUN_BODY>	->	**return** <VALUE>	{ **return**, **if**, **while**, **let**, **var**, **id** }	{ **}** }
25.	<FUN_BODY>	->	<STAT> <FUN_BODY>		
26.	<ASSIGN>	->	ε	{ **=**, **ε** }	{ **EOL** }
27.	<ASSIGN>	->	= <VALUE>		
28.	<IF_H>	->	(<EXPR>)	{ **let**, **(** }	{ **{** }
29.	<IF_H>	->	**let** id		
30.	<VAR_DEC>	->	**let**	{ **let**, **var** }	{ **id** }
31.	<VAR_DEC>	->	**var**		
32.	<DATA_TYPE>	->	**Int** <Q_MARK>	{ **Int**, **Double**, **String** }	{ **=**, **,** , **{**, **)**, **EOL** }
33.	<DATA_TYPE>	->	**Double** <Q_MARK>		
34.	<DATA_TYPE>	->	**String** <Q_MARK>		
35.	<FUN_TYPE>	->	-> <DATA_TYPE>	{ **->**, **ε** }	{ **{** }
36.	<FUN_TYPE>	->	ε		
37.	<Q_MARK>	->	?	{ **?**, **ε** }	{ **=**, **,** , **{**, **)**, **EOL** }
38.	<Q_MARK>	->	ε		
39.	<VALUE>	->	**INT_VALUE**	{ **id**, **nil**, **INT_VALUE**, **DOUBLE_VALUE**, **STRING_VALUE**, **<EXPR>** }	{ **EOL**, **,** , **)** }
40.	<VALUE>	->	**DOUBLE_VALUE**		
41.	<VALUE>	->	id <ARGS>		
42.	<VALUE>	->	**STRING_VALUE**		
43.	<VALUE>	->	**nil**		
44.	<VALUE>	->	<EXPR>		
45.	<VAR_DEF>	->	: <DATA_TYPE> <ASSIGN>	{ **:**, **=** }	{ **EOL** }
46.	<VAR_DEF>	->	= <VALUE>		
47.	<P_VALUE>	->	<VALUE>	{ **id**, **nil**, **INT_VALUE**, **DOUBLE_VALUE**, **STRING_VALUE**, **<EXPR>** }	{ **,** , **)** }
48.	<P_VALUE>	->	id : <VALUE>		

Tabulka 2, LL tabulka

	func	id	(	)	{	}	EOL	EOF	if	else	while	:	=	,	_	return	var	let	Int	Double	String	?	->	INT_V	DOUBLE_V	STRING_V	nil	*exp*
<START>	2	1					3	4	1		1						1	1										
<STAT>		8							5		6						7	7										
<FUN_VAR>			10										9															
<ARGS>			12	11			11							11														
<ARGS2>				13										14														
<STAT_LIST>		17				15			17		17					16	17	17										
<PL>		19		18											19													
<PL2>				20										21														
<TERM>		23													22													
<FUN_BODY>		25							25		25					24	25	25										
<ASSIGN>							26						27															
<IF_H>			28															29										
<VAR_DEC>																	31	30										
<DATA_TYPE>																			32	33	34							
<FUN_TYPE>					36																		35					
<Q_MARK>				38	38		38						38	38								37						
<VALUE>		41																						39	40	42	43	44
<VAR_DEF>												45	46															
<P_VALUE>		47, 48																						47	47	47	47	47

Tabulka 3, Precedenční tabulka

	+, -	***, /**	**rel**	**(**	**)**	**??**	**!**	**id**	**$**
+, -	>	<	>	<	>	>	<	<	>
***, /**	>	>	>	<	>	>	<	<	>
rel	<	<		<	>	>	<	<	>
(	<	<	<	<	=	<	<	<	
)	>	>	>		>	>	>		>
??	<	<	<	<	>	<	<	<	>
!	>	>	>		>	>			>
id	>	>	>		>	>	>		>
$	<	<	<	<		<	<	<	